

Linux Class 10 - Task

Printing Chosen Lines of a File with sed

Submitted by: Shivakumar Muradi

Date: 02 October 2026

For this task I made a folder called class10 and typed a file called lines.txt in vi. It has ten lines and I wrote the number of each line in words, so line one, line two and so on. I did that on purpose, because then I can see straight away whether sed picked the right lines or not.

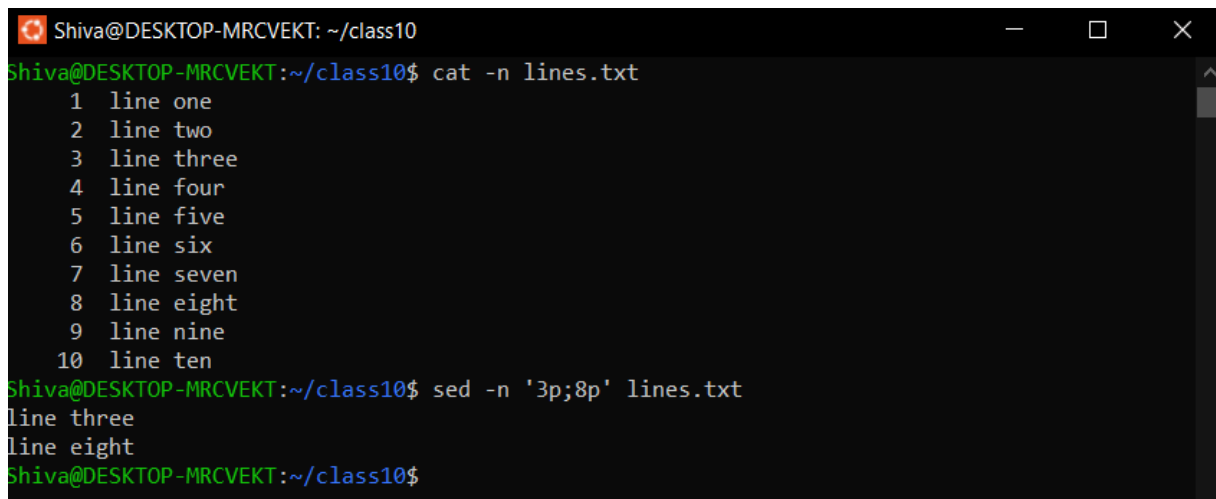
A closed file means a file I have not opened in vi or any other editor. sed reads the file from the command line and prints what I ask for on the terminal, so the file is never opened and never changed.

How to print the 3rd line and the 8th line of a closed file using sed

The command is:

```
sed -n '3p;8p' lines.txt
```

When I ran it, it printed line three and line eight and nothing else, which is exactly what was asked.



```
Shiva@DESKTOP-MRCVEKT: ~/class10
Shiva@DESKTOP-MRCVEKT:~/class10$ cat -n lines.txt
 1 line one
 2 line two
 3 line three
 4 line four
 5 line five
 6 line six
 7 line seven
 8 line eight
 9 line nine
10 line ten
Shiva@DESKTOP-MRCVEKT:~/class10$ sed -n '3p;8p' lines.txt
line three
line eight
Shiva@DESKTOP-MRCVEKT:~/class10$
```

Screenshot 1: the file with its line numbers, and then only lines 3 and 8 printed by sed.

I used cat -n lines.txt first so that the real line numbers are visible in the screenshot. That way anybody looking at it can check that the two lines sed printed really are the 3rd and the 8th.

This is what each part of the command does:

Part	What it does
sed	The command itself.
-n	Do not print anything on its own. Without this, sed prints every line.
'...'	The single quotes hold the instructions, so the shell leaves them alone.
3p	Print line 3. The number is the line and p means print.
;	Separates one instruction from the next.
8p	Print line 8.
lines.txt	The file to read. It is never changed.

Why the -n is needed

The -n is the part I did not understand at first. sed normally prints every line it reads, whether I asked for it or not. So if I leave out the -n, the p instruction does not replace that printing, it adds to it:

```
sed '3p;8p' lines.txt
```

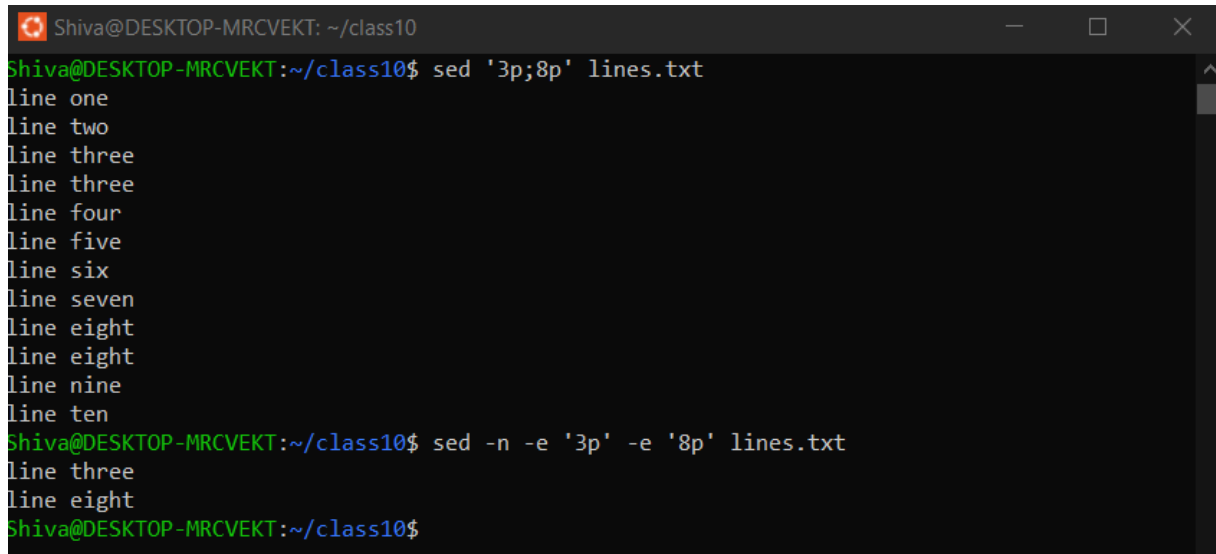
This printed the whole file of ten lines, and lines 3 and 8 came out twice each, so the output had twelve lines. sed printed them once because of p and once more because that is what it always does.

So -n stands for no automatic printing. With -n the only lines that appear are the ones I asked for with p. This is the mistake I made the first time.

There is also another way to write the same command, giving each instruction its own -e:

```
sed -n -e '3p' -e '8p' lines.txt
```

It gives exactly the same answer. The -e simply means here comes an instruction, so I can list several of them instead of joining them with a semicolon.



```
Shiva@DESKTOP-MRCVEKT: ~/class10
Shiva@DESKTOP-MRCVEKT:~/class10$ sed '3p;8p' lines.txt
line one
line two
line three
line three
line four
line five
line six
line seven
line eight
line eight
line nine
line ten
Shiva@DESKTOP-MRCVEKT:~/class10$ sed -n -e '3p' -e '8p' lines.txt
line three
line eight
Shiva@DESKTOP-MRCVEKT:~/class10$
```

Screenshot 2: without -n every line is printed and lines 3 and 8 appear twice. The -e form gives the right answer.

The semicolon and the comma are not the same

This is the part that is easy to get wrong. A semicolon joins two separate instructions, but a comma makes a range:

```
sed -n '3,8p' lines.txt
```

This printed six lines, from line three all the way down to line eight, because the comma means from 3 to 8. It is a completely different answer from the one the question asks for.

- 3p;8p means line 3 and line 8.
- 3,8p means line 3 through to line 8.

One more useful option is q, which means quit. Since there is nothing I need after line 8, I can tell sed to stop reading there:

```
sed -n '3p;8p;9q' lines.txt
```

It gives the same two lines. On a small file like mine it makes no difference, but on a very big file it saves sed from reading thousands of lines it does not need.

The same job can also be done with awk:

```
awk 'NR==3 || NR==8' lines.txt
```

NR is the line number and the two vertical bars mean or, so it prints the line when the line number is 3 or 8.

```
Shiva@DESKTOP-MRCVEKT: ~/class10
Shiva@DESKTOP-MRCVEKT:~/class10$ sed -n '3,8p' lines.txt
line three
line four
line five
line six
line seven
line eight
Shiva@DESKTOP-MRCVEKT:~/class10$ sed -n '3p;8p;9q' lines.txt
line three
line eight
Shiva@DESKTOP-MRCVEKT:~/class10$ awk 'NR==3 || NR==8' lines.txt
line three
line eight
Shiva@DESKTOP-MRCVEKT:~/class10$
```

Screenshot 3: the comma gives a whole range, then the correct answer with q, and the same answer with awk.

All the forms together

Command	What it gives
<code>sed -n '3p;8p' lines.txt</code>	Line 3 and line 8 only. This is the answer.
<code>sed -n -e '3p' -e '8p' lines.txt</code>	The same thing, with each instruction given its own -e.
<code>sed '3p;8p' lines.txt</code>	The whole file, with lines 3 and 8 printed twice.
<code>sed -n '3,8p' lines.txt</code>	Lines 3 to 8 together. The comma makes it a range.
<code>sed -n '3p;8p;9q' lines.txt</code>	The same answer, but sed stops reading at line 9.
<code>sed -n '\$p' lines.txt</code>	The last line, whatever its number is.
<code>awk 'NR==3 NR==8' lines.txt</code>	The same answer using awk instead.

Everything above only prints on the terminal. None of these commands change lines.txt, and I never opened the file in an editor. If I did want to save the result I would send it to another file, but that was not part of the question.